**Task ‘Unit 4 Design Patterns II - Structural Patterns’ Unit 4**

**Task Collaborative Discussion 1**

Structural design patterns helps to improve maintainability and flexibility. The Adapter Pattern helps incompatible systems communicate, for instance, in a payment scenario, an adapter converts REST JSON requests into SOAP messages for a legacy gateway, this avoids rewriting the old system.

(Lott and Phillips, 2021)

```python
class LegacyPay:
    def pay_soap(self, data): print("SOAP:", data)


class Adapter:
    def __init__(self, legacy): self.legacy = legacy
    def pay_json(self, data): self.legacy.pay_soap(str(data))


Adapter(LegacyPay()).pay_json({"amount":100})
```

The Bridge Pattern separates abstraction from implementation, the  remote control, can work with different devices without changing its code, this reduces duplication and supports scalability.

```python
class TV:
    def on(self): print("TV on")


class Radio:
    def on(self): print("Radio on")


class Remote:
    def __init__(self, device): self.device = device
    def power(self): self.device.on()


Remote(TV()).power()
```

The Composite Pattern treats files and folders equally, for example, it simplifies recursive operations such as displaying or searching a directory tree by treating folders containing files or other folders equally.

```python
class File:
    def __init__(self, name): self.name=name
```

```
    def show(self): print(self.name)


class Folder:
    def __init__(self, name):
        self.name=name; self.items=[]
    def add(self, item): self.items.append(item)
    def show(self):
        print(self.name)
        for i in self.items: i.show()


docs=Folder("Docs")
docs.add(File("notes.txt"))
docs.show()
```

(Refactoring Guru, 2024)

Using the above techniques, help to develop a better and a more reusable design that improves readability, extensibility and code maintenance, with the Adapter enabling better integration, Bridge producing independent variations and Composite simplifying hierarchical structures.

(Lott and Phillips, 2021)

## References


- Steven F. Lott, D. P. (2021) Python Object-Oriented Programming: Build robust and maintainable object-oriented Python applications and libraries, 4th Edition. Packt Publishing.
- Refactoring Guru (2024) Adapter Pattern. Refactoring Guru Adapter Pattern, available at: https://refactoring.guru/design-patterns/adapter [Accessed: 16 May 2026].
- Refactoring Guru (2024) Bridge Pattern. Refactoring Guru Bridge Pattern, available at: https://refactoring.guru/design-patterns/bridge [Accessed: 16 May 2026].
- Refactoring Guru (2024) Composite Pattern. Refactoring Guru Composite Pattern, available at: https://refactoring.guru/design-patterns/ composite [Accessed: 16 May 2026].


## Use of Digital Tools

This document has been produced exclusively for educational purposes. Referenced content, copyrights, and trademarks, remain the sole property of the respective owner. ChatGPT 5.5 was used for some proofreading and clarity as well as literature exploration, language refinement, coding and programming techniques research. All academic writing, analysis, argumentation, and conclusions represent the original work of the author. AI tools were used for exploration, same way as academic literature exploration. No AI-generated content or code was directly copied into the final submission. Any use of content mentioned in this document is properly referenced to its original author and was used responsibly to ensure integrity and originality. I have used Artificial Intelligence responsibly and ethically, in accordance with the assignment brief. All AI use has been declared and evidenced.